

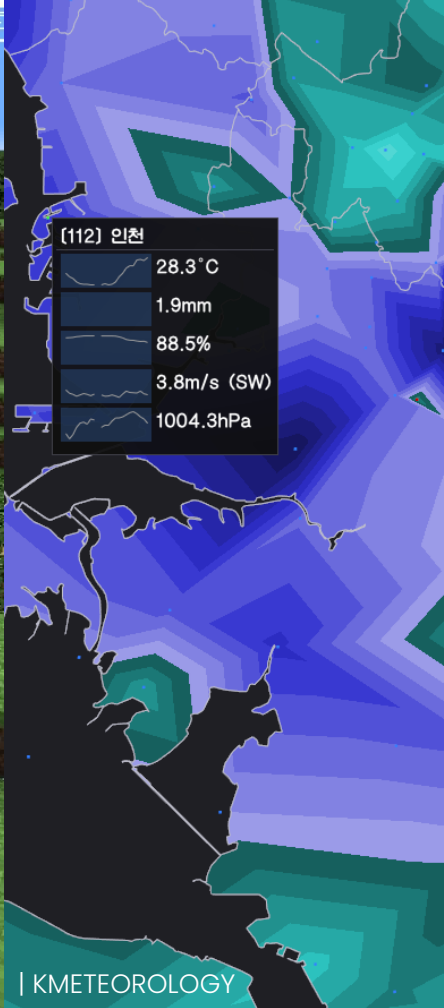
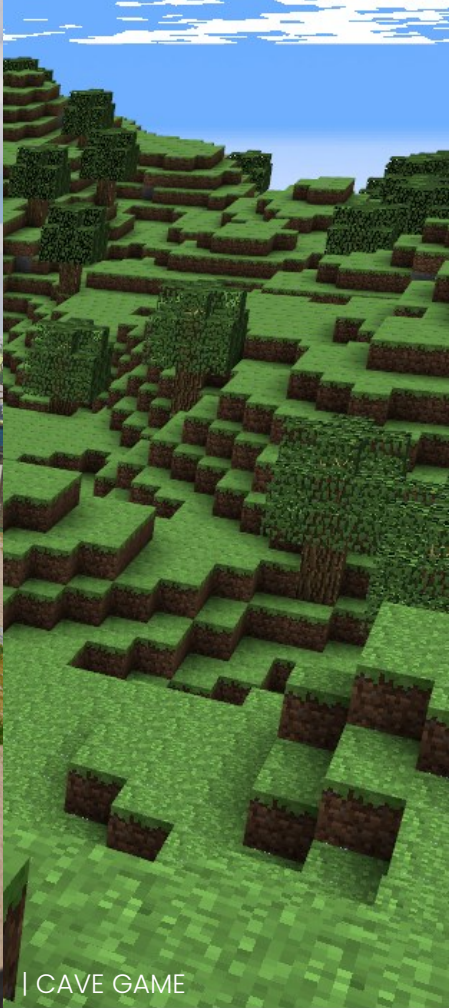
그래픽스 프로그래밍 포트폴리오

김도엽

Tel. : 010 - 2435 - 3404

E-Mail : do.yupdown@gmail.com





| EVERGREEN

| CAVE GAME

| KMETEOROLOGY

PROJECTS

EVERGREEN

게임공학과 졸업작품

- 프로젝트 개요
- Development Showreel
- DirectX 12 Framework
- Deferred HDR Render
- Discrete Materials With Deferred Render
- Per-Object Motion Blur
- Physically Based Bloom
- CSM
- Level Export With Unity Editor

CAVE GAME

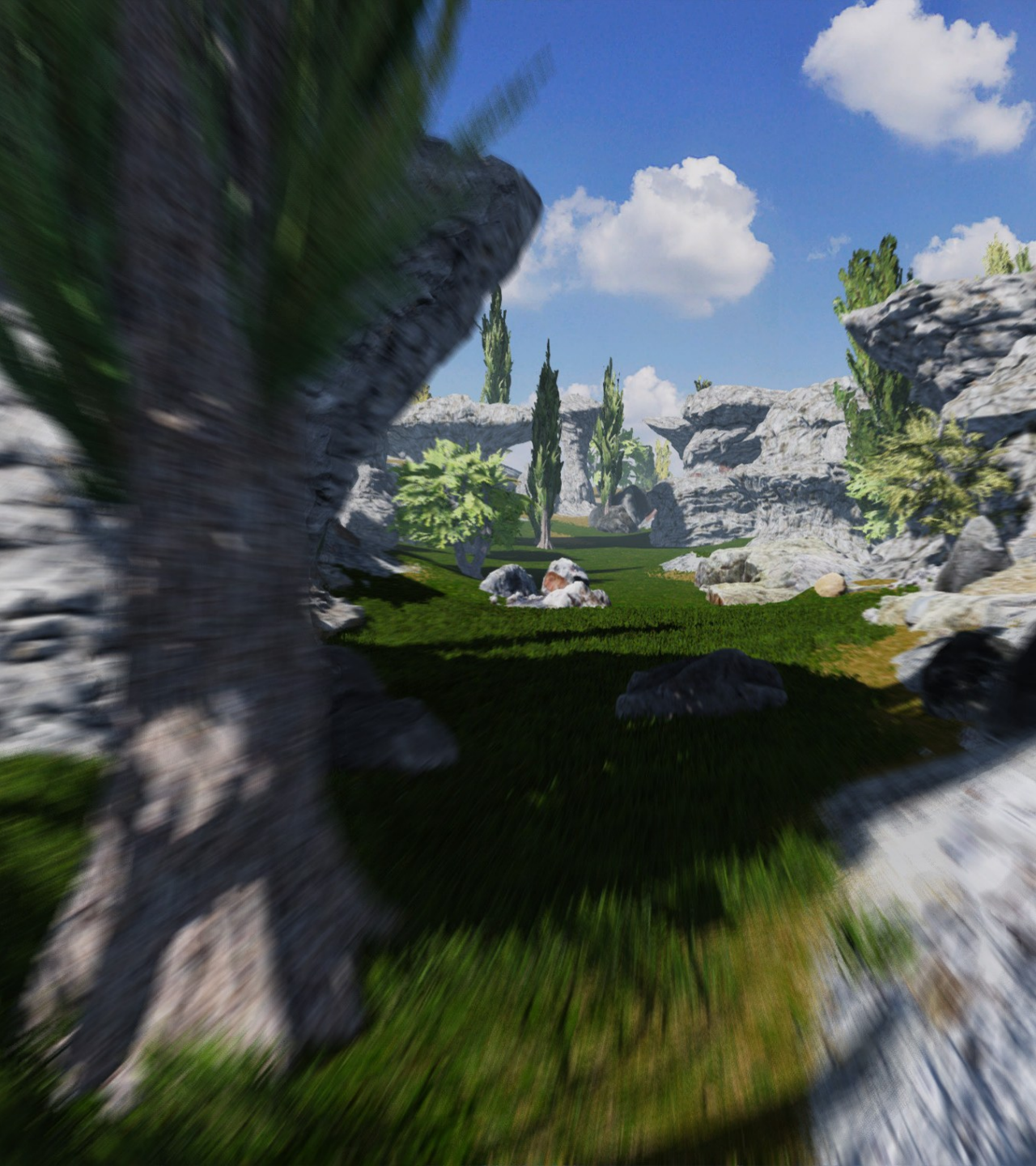
컴퓨터그래픽스 텀프로젝트

- 프로젝트 개요
- 절차적 지형 생성
- 렌더링 최적화

KMETEOROLOGY

스크립트언어 텀프로젝트

- 프로젝트 개요
- 벡터 기반 지도
- 히트맵 렌더링



2025 한국공학대전 한국산업기술기획평가원 원장상 수상작

EVERGREEN

프로젝트 개요



프로젝트 이름

Evergreen (에버그린)



장르

오픈 월드 판타지 MMORPG

설명

파티퀘스트를 통한 협동 플레이를 중심으로 넓은 세계를 자유롭게 탐험하며 채집 및 사냥하는 게임

개발 인원

3명 (서버 프로그래밍, 클라이언트 프로그래밍, 캐릭터 모델링)

담당 역할

클라이언트 프로그래밍 - 렌더러 프레임워크 구성, 셰이더 작성, 레벨 파싱, 이펙트, 인터페이스

개발 언어

C++20, HLSL

사용 툴 & 라이브러리

Visual Studio 2022, DirectX 12, Assimp, ImGui, Nsight Graphics, Git

제작 기간

2024 / 03 ~ 2025 / 09

EVERGREEN

DEVELOPMENT SHOWREEL



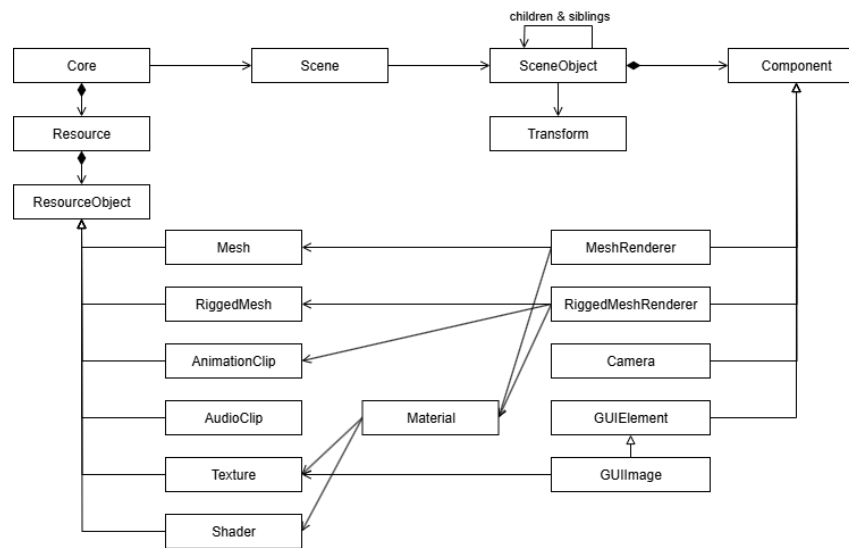
<https://youtu.be/oG4O6Fxhq7A>

DIRECTX 12 FRAMEWORK

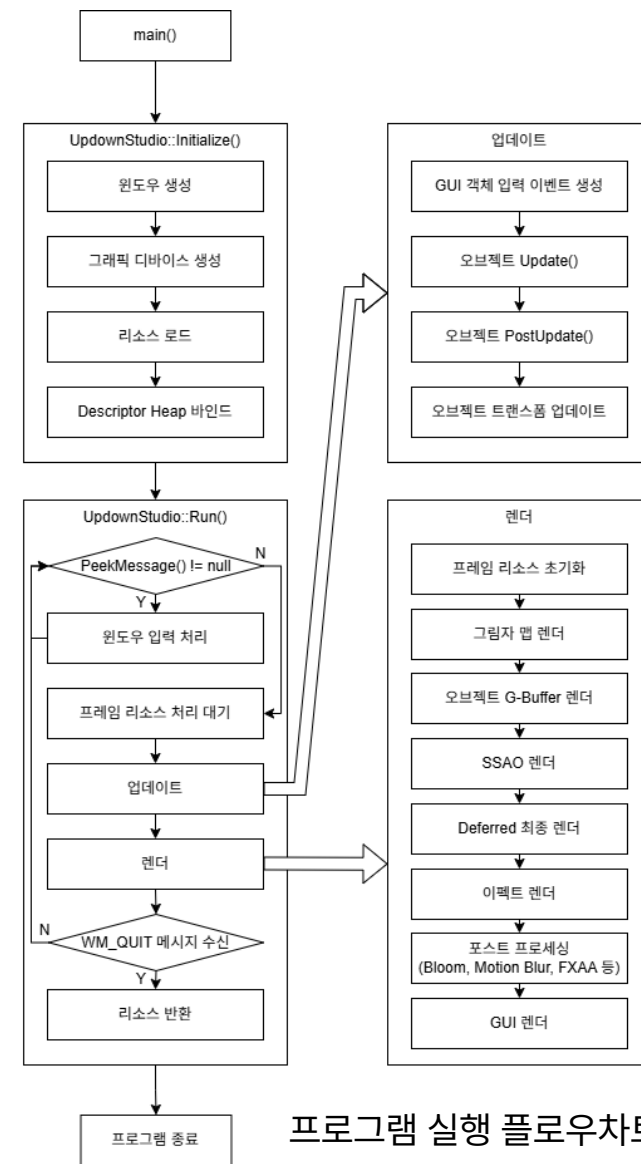
- ECS (Entity Component System) 기반 DirectX 12 프레임워크
- SceneObject는 월드 트랜스폼 정보를 가지며, AddChild(), RemoveFromParent() 함수로 다른 SceneObject와 계층적인 트리 구조를 구성할 수 있음
- SceneObject는 임의의 Component를 추가할 수 있으며, Component의 생명 주기에 따라 OnAttach(), Update(), PostUpdate(), OnDetach() 등의 이벤트를 자동으로 호출 [\[참고\]](#)

? 매 프레임마다 월드 트랜스폼을 재연산하는 과정에서 연산이 필요없는 정적인 오브젝트도 불필요한 행렬곱 연산이 발생하는것을 어떻게 해결하는가?

→ 더티 플래그 패턴을 이용하여, 해당 프레임에서 자신과 부모의 오브젝트에 로컬 트랜스폼 변경이 일어나지 않았다면 연산을 생략하는 식으로 해결 [\[참고\]](#)



프레임워크 클래스 다이어그램



프로그램 실행 플로우차트

DIRECTX 12 FRAMEWORK

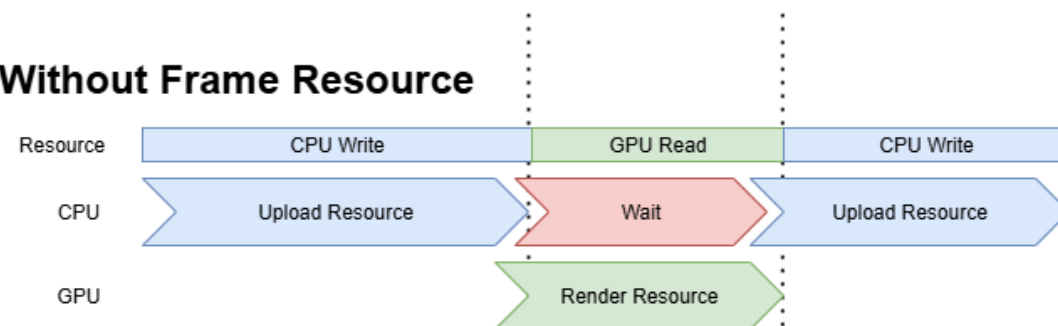
? CPU와 GPU는 병렬로 작동한다. GPU가 현재 프레임의 정보를 렌더하는 동안 CPU가 다음 프레임의 정보로 덮어 쓰려고 하면 비정상적인 출력이 발생하고, 이를 방지하기 위해 CPU는 GPU가 렌더를 끝낼 때 까지 기다려야 한다. CPU가 GPU의 렌더를 기다리지 않고 다음 프레임 정보에 대한 작업을 수행할 수 있게 할 수 있는가?

→ “Frame Resource” 개념을 적용하여, 2개 이상의 프레임에 대한 정보를 각각 저장할 수 있는 공간을 따로 마련하고 GPU가 현재 렌더하고 있지 않은 프레임의 정보를 CPU가 덮어쓰게 하는 방식으로 해결

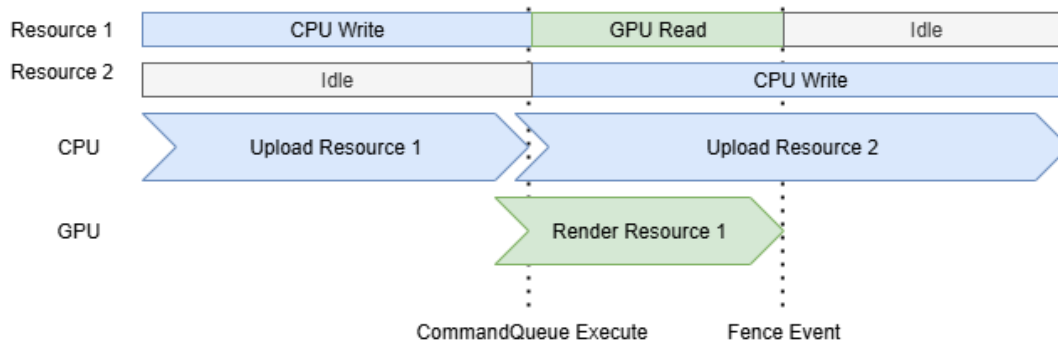
? 셰어드 포인터로 관리되고 있는 SceneObject는 씬의 업데이트 루프 내에서 사용자에게 의해 어느 시점에서든 추가 또는 삭제될 수 있다. 렌더가 예정된 오브젝트가 곧바로 삭제되면 반환된 메모리의 정보를 렌더하게 될 수도 있다. 이를 방지할 수 있는 방법이 있는가?

→ 셰어드 포인터의 Deleter를 재정의하여, 인스턴스를 바로 삭제하지 않고 GarbageCollector에 보관한 후 해당 인스턴스의 렌더를 마친 경우에 메모리 반환을 수행하는 방식으로 해결 [문](#) [문](#)

Without Frame Resource



With Frame Resource

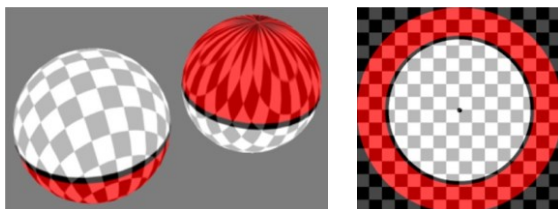


Frame Resource 적용 전 / 후 플로우차트

DEFERRED HDR RENDER

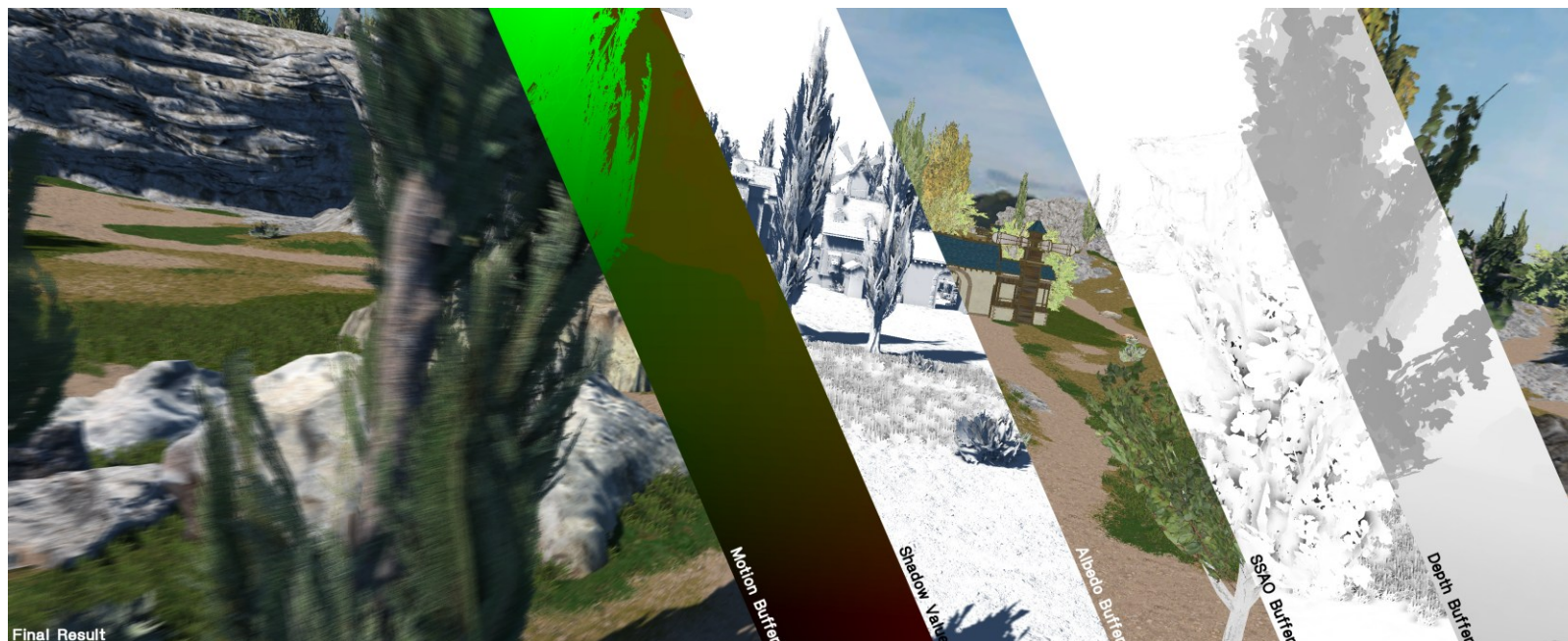
- 3개의 G-Buffer를 이용하여 Deferred HDR Render 적용
- 최종 패스 이전 G-Buffer의 정보를 활용하여 Screen Space Ambient Occlusion 적용 [☞](#)
- 최종 패스에서 픽셀의 월드 위치는 카메라 변환 행렬과 Depth 버퍼의 깊이값으로 추측
- ? G-Buffer에 노멀 정보를 더 높은 정확도로 저장하려고 한다.

→ LAEAP Lambert azimuthal equal-area projection 을 이용하여, XYZ인 노멀을 XY 으로 인코딩 후 R16G16 버퍼에 저장 [☞](#)



LAEAP 노멀 인코딩

	Format	Usage			
		R	G	B	A
RT0	R8G8B8A8_UNORM	Albedo			
RT1	R16G16_SNORM	Normals (Encoded)		Unused	
RT2	R8G8B8A8_SNORM	Motion (Screen Space)		Metallic	Roughness



DISCRETE MATERIALS WITH DEFERRED RENDER

? 캐릭터는 툰 그림자를 나타내는 NPR 렌더링을 수행하는 동시에, 배경에는 PBR 렌더링을 수행하고 싶다. 이를 추가적인 Forward Render 없이 Deferred Render 만으로 어떻게 달성하는가?

→ Stencil Buffer에 셰이더 고유 ID(최대 128개) 저장 후, 최종 패스에 ID 마다 Stencil Test를 적용하여 셰이더별로 차례로 최종 결과 렌더 [문](#)

- DirectX 12 그래픽스 파이프라인에서 Stencil에 대한 Early-Z를 수행하기에, 최종 패스에서 각각의 셰이더별 영역만큼 픽셀 연산 수행
- 스텐실 버퍼에 외곽선에 대한 부울 정보를 0x80에 추가하여, 후처리에서 Convolution을 이용한 외곽선 효과 적용 [문](#)



Outline Effect



Stencil (S8_UINT)

Final render with stencil test



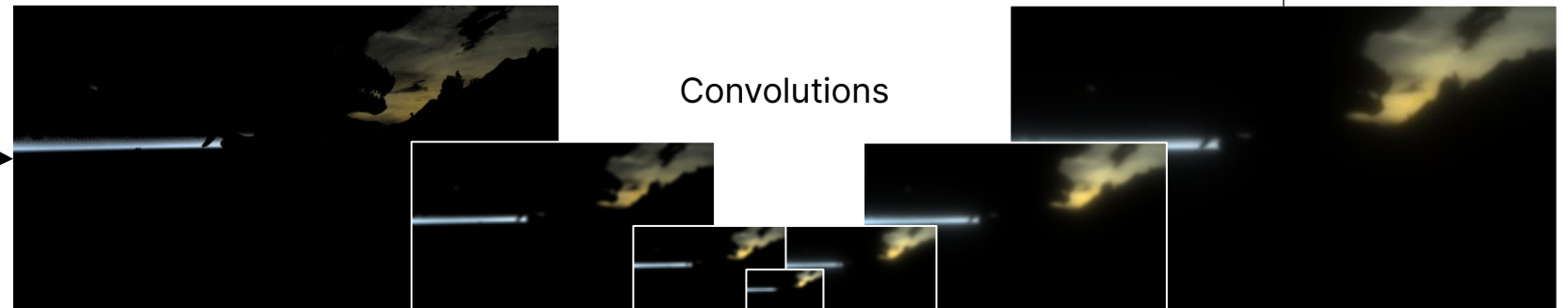
PER-OBJECT MOTION BLUR

- 프레임 간 시간적인 아티팩트를 해결하기 위해 카메라 움직임, 오브젝트 이동 · 회전 · 크기 변화, 캐릭터 애니메이션에 모션 블러 적용
- 이전 프레임의 오브젝트 트랜스폼, 애니메이션 트랜스폼, 카메라 변환 정보를 임시로 저장한 후 화면 공간에서 변위를 구하여 G-Buffer에 저장
- 후처리 단계에서 변위 정보를 활용하여 가우시안 블러 적용. [\[4\]](#)
- ["A reconstruction filter for plausible motion blur"](#) 논문 참조



PHYSICALLY BASED BLOOM

- 밝은 영역에 대한 현실적인 효과를 구성하기 위해 물리 기반 블룸 적용
- ? 물리 기반 블룸을 적용하기 위해서는 매우 넓은 영역에 대한 Convolution이 필요하여 큰 GPU 오버헤드를 발생시킨다. 이를 어떻게 완화하는가?
 - R11G11B10_FLOAT 로 저장된 중간 결과 화면의 밝은 영역을 수집하여, 지수 크기를 가진 여러 개의 버퍼에 걸쳐 downsample, upsample을 적용하여 밝은 영역을 주변으로 확산 ([SIGGRAPH 2014](#))
- 확산된 밝은 영역에 대한 버퍼를 결과 화면에 블렌딩 함과 동시에 SEUS Tone Mapping 적용 [문](#)



CSM CASCADED SHADOW MAPS

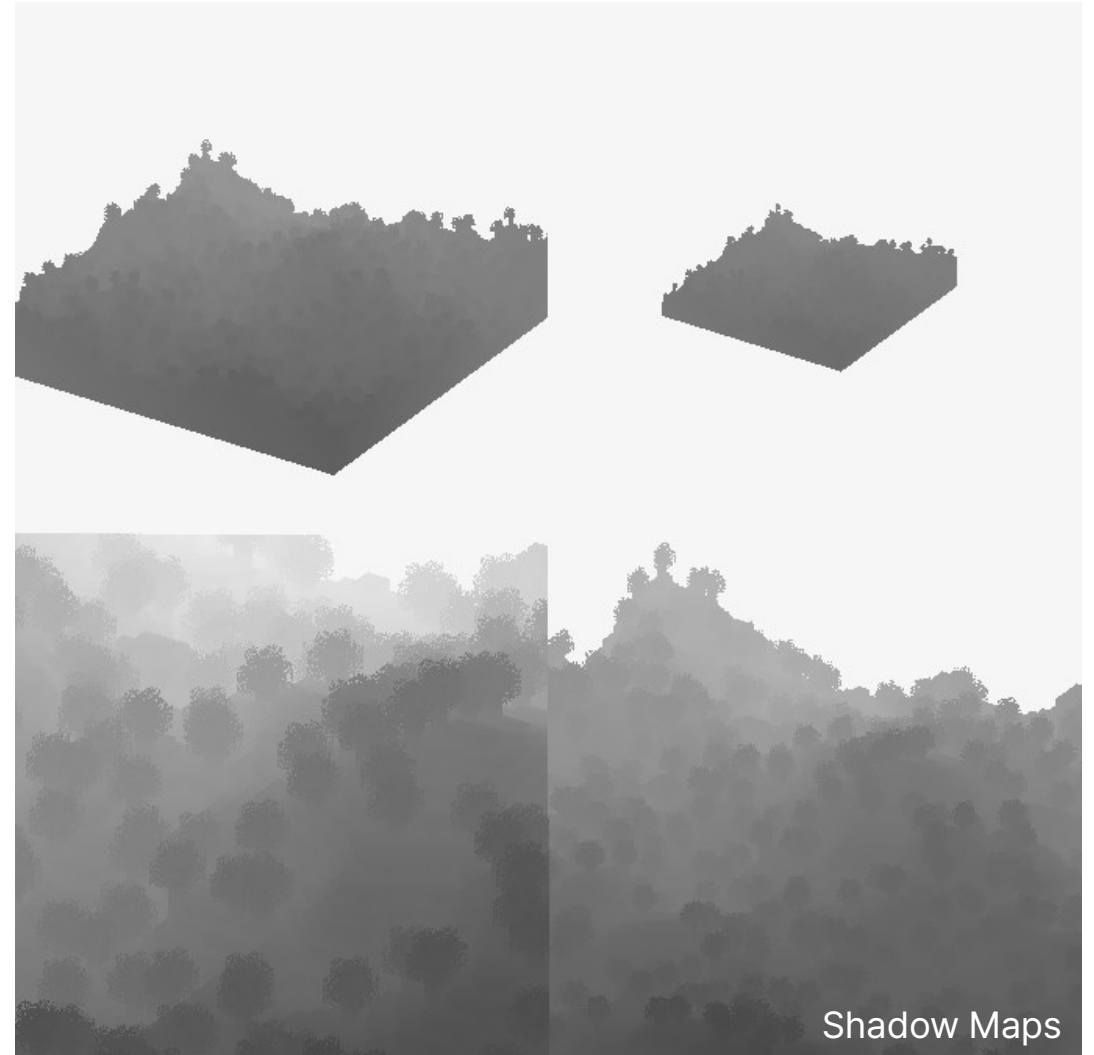
? 넓은 맵을 바라보는 3인치 자유 시점 특성상, 그림자의 해상도 - 거리 간 Trade-Off가 불가피하다. 이를 어떻게 완화하는가?

→ 4-Levels CSM을 적용하여 가까운 곳은 해상도가 높은 그림자 맵을 사용하면서, 먼 거리까지 그림자를 렌더링

- 거리에 따른 그림자 levels 간 경계를 lerp 함수를 통해 부드럽게 보간 [☞](#)
- 부드러운 그림자 경계를 위해 PCF Percentage Closer Filtering 적용 [☞](#)

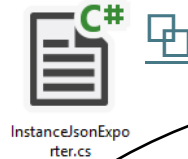


근경 · 원경 그림자 매핑 결과 (그림자 맵: 4096 × 4096 px)



LEVEL EXPORT WITH UNITY EDITOR

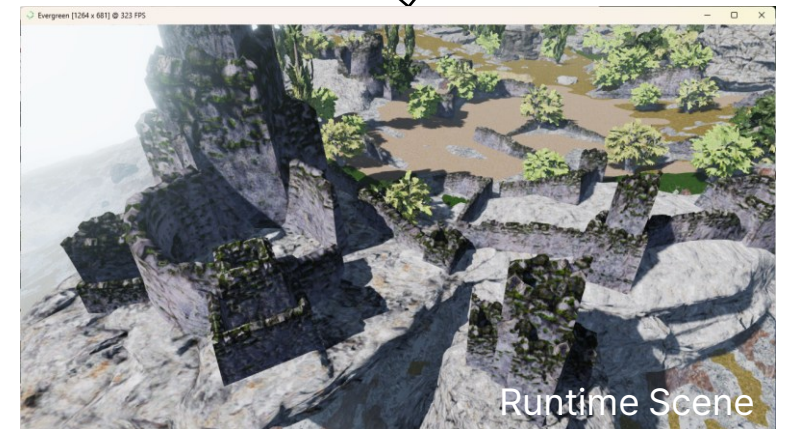
- 지형, 월드 오브젝트, 카메라 스플라인 등의 제작 배치를 원활하기 수행하기 위해 유니티 에디터를 활용
- 유니티 에디터에서 작업한 씬을 직접 C#으로 제작한 Exporter를 통해 .json으로 변환하여 프로그램에 파일로써 입력
- 월드 오브젝트 배치 정보는 씬에서 각각의 모델에 대한 인스턴스 리스트를 .json으로 추출하여, 각각의 트랜스폼 정보를 입력으로 받은 후 인스턴싱



```
{
  "prefab": "0_Addon_SmallWindow_D",
  "instances": [
    {
      "position": {
        "x": -29.85693359375,
        "y": 82.844970703125,
        "z": -121.6959228515625
      },
      "rotation": {
        "x": 2.1855694143368966e-8,
        "y": 0.0,
        "z": 0.0,
        "w": 1.0
      },
      "scale": {
        "x": 1.0,
        "y": 1.0,
        "z": 1.0
      }
    },
    {
      "position": {
        "x": -27.3795166015625,
        "y": 82.844970703125,
        "z": -121.6959228515625
      },
      "rotation": {
        "x": 2.1855694143368966e-8,
```



Unity Scene



Runtime Scene



2024 한국공학대학교 게임공학과 과제전 종합우승작

CAVE GAME



CAVE GAME

프로젝트 개요



프로젝트 이름

Cave Game



장르

오픈 월드 샌드박스 게임

설명

상용 게임 "Minecraft"를 재구현하여 절차적으로 생성된 지형에서 블록을 설치하고 파괴할 수 있는 게임

개발 인원

2명 (클라이언트 프로그래밍)

담당 역할

지형 생성, 셰이더 작성, 게임플레이 프로그래밍

개발 언어

C++20, GLSL

사용 툴 & 라이브러리

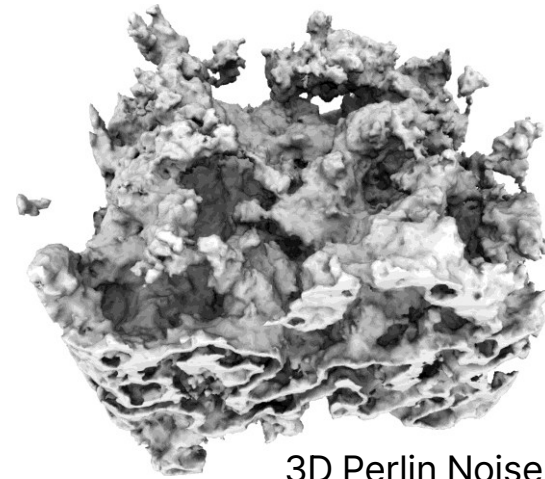
Visual Studio 2022, OpenGL, Assimp

제작 기간

2023 / 09 ~ 2023 / 12

절차적 지형 생성

- $1 \times 1 \times 1$ 크기의 정육면체로 이루어진 블록의 집합으로 Voxel 기반 지형 구성
- 여러 단계들로 이루어진 지형 생성 알고리즘을 통해 평지, 언덕, 동굴 등의 다양한 지형을 생성 가능. 절차적 지형 생성에는 3D Perlin Noise 알고리즘 등을 사용 [\[참고\]](#)
- 각각의 타일은 고유한 텍스처를 가지며, 플레이어가 카메라를 향한 방향의 타일을 Ray Casting 알고리즘을 통해 설치하거나 파괴 가능 [\[참고\]](#)
- 플레이어를 포함한 모든 오브젝트는 지형과 충돌 시뮬레이션. 3차원 배열로 저장되어진 블록의 정보를 오브젝트의 3차원 위치를 통해 상수 시간복잡도로 접근하여 적은 오버헤드 [\[참고\]](#)



3D Perlin Noise



절차적으로 생성된 지형

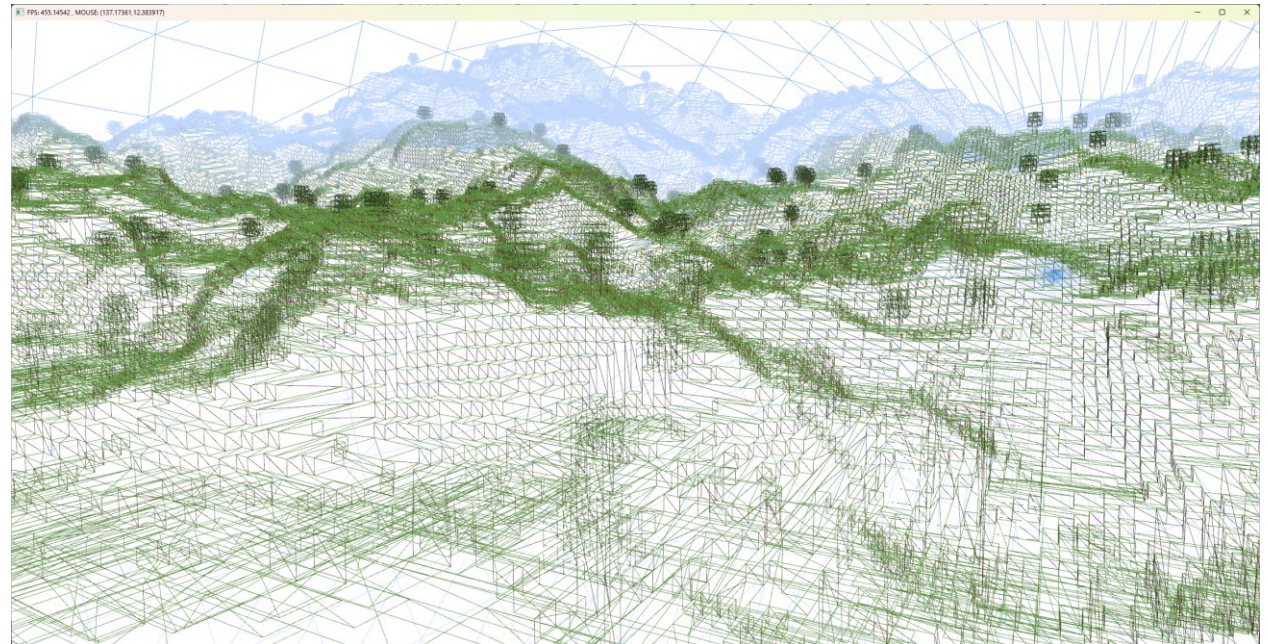
CAVE GAME

렌더링 최적화

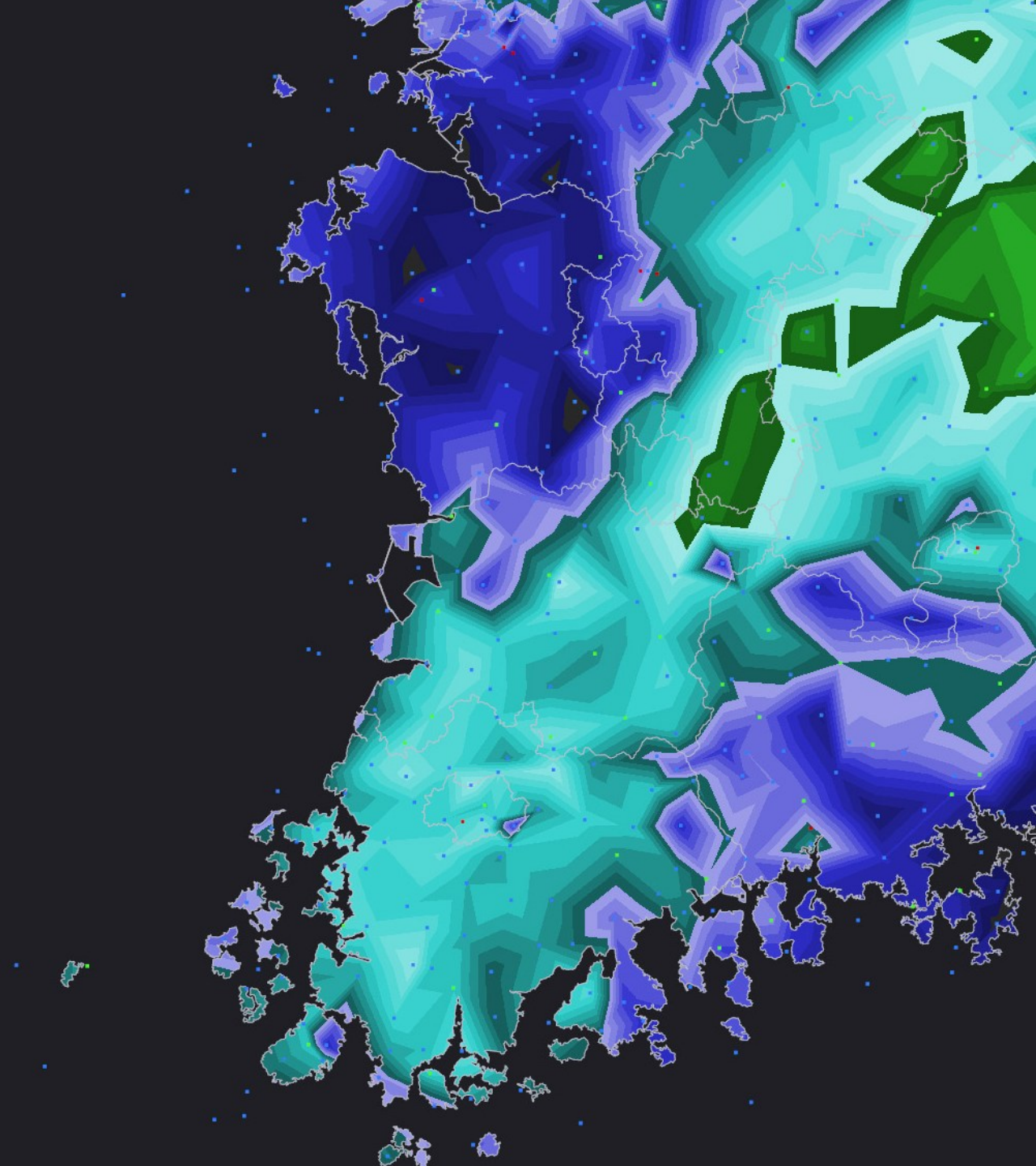
- ? Cave Game에서의 블록의 개수는 최대 월드 길이의 세제곱이 될 수 있다. 모든 블록의 메쉬를 각각 생성하면 매우 심각한 렌더링 오버헤드가 발생할 것이다.
 - 지형에서 보이지 않는 면은 버텍스 / 인덱스 버퍼 생성 단계에 폴리곤을 생성하지 않음으로써, 오버드로우 현상을 최소화 [\[문\]](#)
- ? 블록을 설치하거나 파괴할 때, 버텍스 / 인덱스 버퍼의 정보를 바꾸어야 하는데, 월드의 모든 블록을 순회하는 과정은 프레임 드랍을 유발한다.
 - 월드를 $32 * 32 * 32$ 블록 크기의 "청크"로 나누어 별도의 버퍼로 관리 [\[문\]](#)
- 지형의 드로우 콜을 줄이기 위해 다양한 블록 텍스처들을 하나의 텍스처 아틀라스로 만들어, 버텍스 당 UV의 값 조작으로 여러 텍스처를 한번의 드로우 콜로 렌더링 [\[문\]](#)



통합된 블록 텍스처



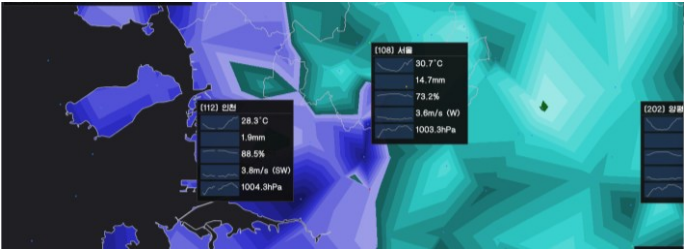
지형 와이어프레임



—

KMETEOROLOGY

프로젝트 개요



프로젝트 이름

KMeteorology



설명

기상청 데이터를 이용한 OpenGL 기반 실시간 기상 데이터 시각화

개발 인원

1명

담당 역할

API 를 통한 데이터 수신 및 파싱
벡터 기반 전국 지도 & 히트맵 렌더러 구성

개발 언어

Python 3.12, GLSL

사용 툴 & 라이브러리

PyCharm, OpenGL, ImGui

제작 기간

2024 / 03 ~ 2024 / 06

벡터 기반 지도

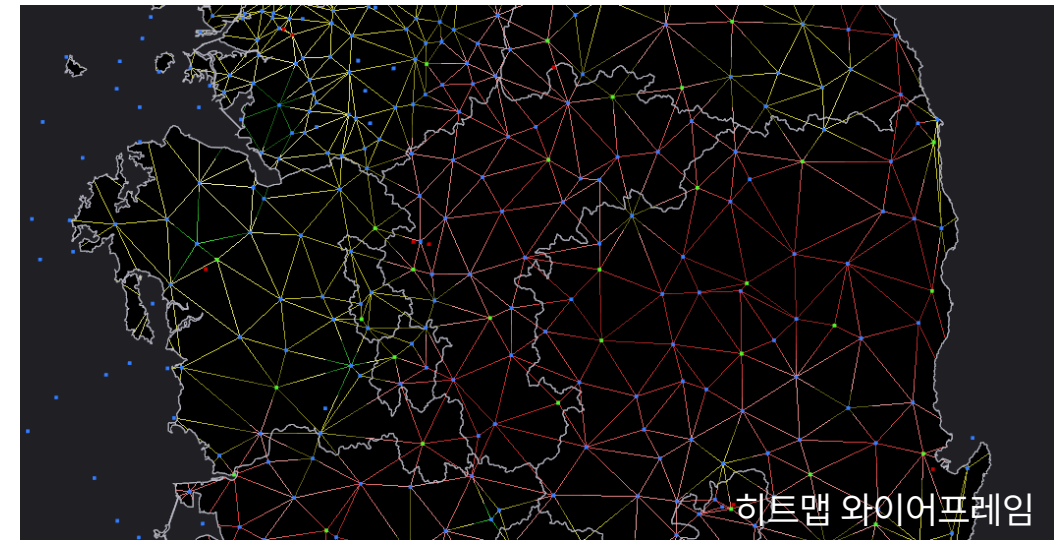
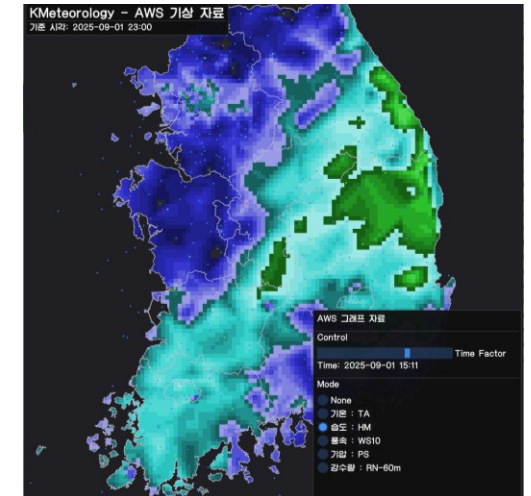
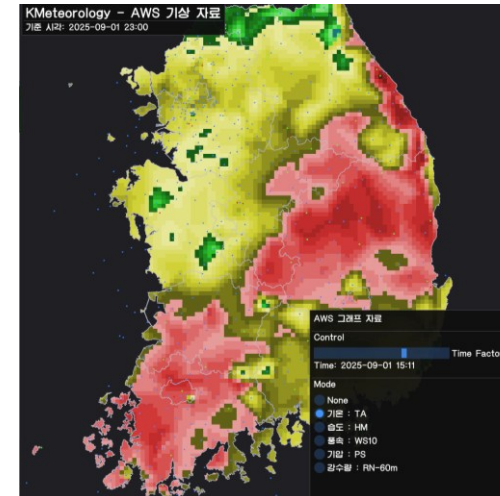
- SVG 형태의 전국 지도를 파싱하여 다각형 정점 리스트를 추출 [\[4\]](#)
- 지도의 각 영역은 단순 다각형 (Simple Polygon) 이라는 점을 활용하여 다각형의 한 변과 임의의 점으로 구성된 삼각형을 그렸을 때, 홀수번 덮어 그려진 영역을 최종적으로 렌더
- 스텐실 버퍼의 GL_INVERT OP를 활용하여 그려질 영역에 대한 스텐실 버퍼를 기록하고, 이후 내부 영역을 스텐실 테스트와 함께 렌더 [\[4\]](#)
- 영역의 외곽선은 GL_LINE_LOOP로 렌더 [\[4\]](#)



히트맵 렌더링

- 각각의 관측소에서 수집된 기상 정보 (기온, 습도, 풍속, 기압, 강수량) 사이의 위치에 대한 정보를 추론 후 시각화하기 위해 주변 3개의 관측소 정보를 선형 보간
- 각 관측소의 위경도를 지도 공간에 매핑한 정점 집합에 대해서 들로네 삼각분할 알고리즘으로 생성된 삼각형에 대한 인덱스 버퍼 생성 [\[원본\]](#)
- 각 관측소에 대응되는 정점에 기상 정보값을 VBO를 통해 GPU에 전송
- 래스터라이저에 보간된 픽셀 위치의 기상 정보값을 1D 컬러 팔레트 텍스처에 매핑하여 히트맵 렌더 [\[원본\]](#)

1D 컬러 팔레트 텍스처



히트맵 와이어프레임



THANK YOU

